

Transformer Paper Implementation

Attention is all you need Paper

References

Youtube video implementation step-by-step:

<https://www.youtube.com/watch?v=ISNdQcPhsts>

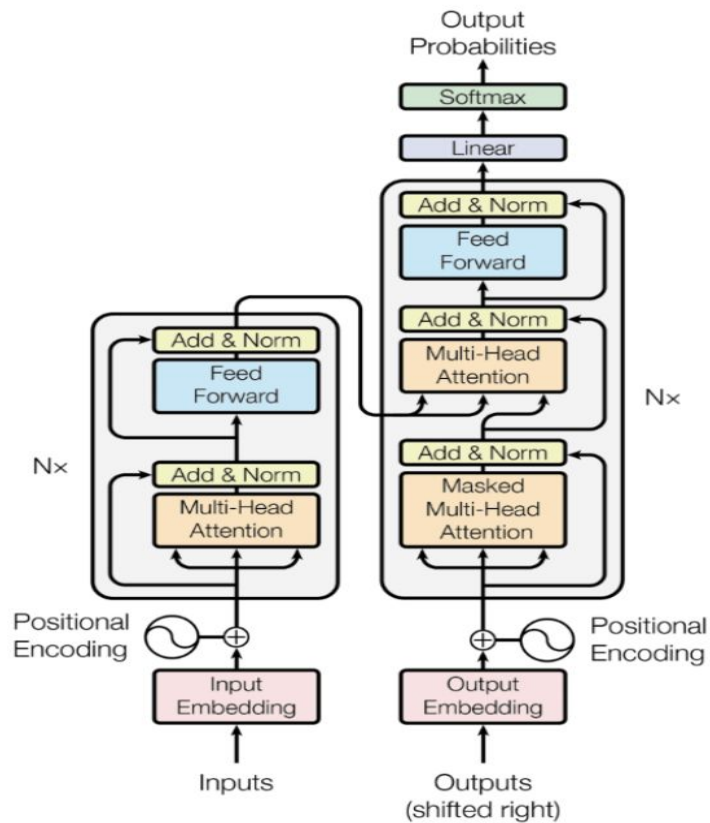
Implementation:

<https://github.com/hkproj/pytorch-transformer/tree/main>

MyRepo:

<https://github.com/Ceachi/pytorch-transformer>

Introducing Transformer



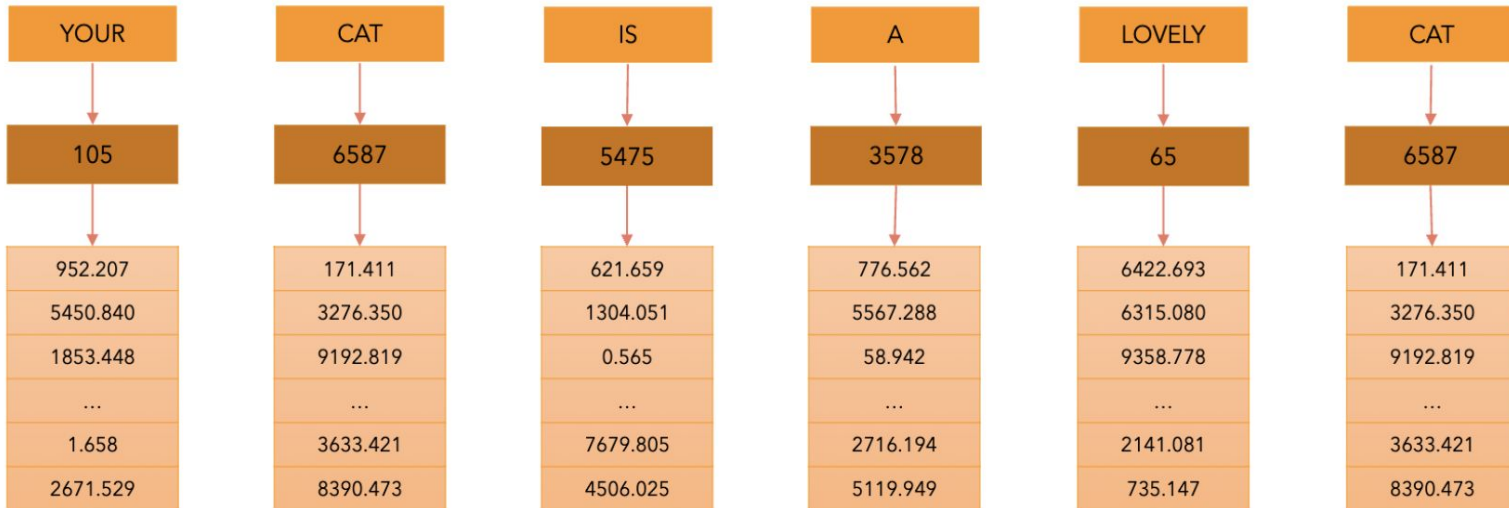
Input matrix (sequence, d_{model})

Encoder

Input Embeddings + Positional Encoding

What is an input embedding?

Original sentence
(tokens)



Input IDs (position in the vocabulary)

Embedding
(vector of size 512)

We define $d_{\text{model}} = 512$, which represents the size of the embedding vector of each word

InputEmbeddings

```
class InputEmbeddings(nn.Module):  
  
    def __init__(self, d_model: int, vocab_size: int) -> None:  
        super().__init__()  
        self.d_model = d_model  
        self.vocab_size = vocab_size  
        self.embedding = nn.Embedding(vocab_size, d_model)  
  
    def forward(self, x):  
        # (batch, seq_len) --> (batch, seq_len, d_model)  
        # Multiply by sqrt(d_model) to scale the embeddings according to the paper  
        return self.embedding(x) * math.sqrt(self.d_model)
```

Explanations in Next slide

InputEmbeddings

- **Purpose:** Transforms discrete token IDs into continuous vectors of size `d_model`.
- **Key Operation:** Uses `nn.Embedding` to look up embeddings, then multiplies by $\sqrt{d_{\text{model}}}$ (as per the original Transformer paper).
- **Input/Output Shapes:**
 - **Input:** (batch_size, seq_len)
 - **Output:** (batch_size, seq_len, d_{model})
- **Why the $\sqrt{d_{\text{model}}}$ scaling?**
 - Ensures embeddings have a stable variance at the start of training.
 - Matches the paper's recommendation for improving optimization.

Explanations in Next slide

InputEmbeddings

- **What is `nn.Embedding` ?**
 - It's a PyTorch layer for mapping each integer token ID (e.g., a word index) to a dense, learned vector of fixed size.
 - It behaves like a dictionary: given a token index, you get back its corresponding embedding vector.
 - These embedding vectors are trained during the model's backpropagation.
- **Why use it?**
 - You don't have to manually define a word vector for each token; they're automatically learned.
 - It's especially common in NLP tasks (e.g., language models, Transformers) where you start with discrete words or subword tokens.

Explanations in Next slide

InputEmbeddings

python

 Copy  Edit

```
import torch
from torch import nn

# Suppose we have 10 tokens in the vocabulary and we want embeddings of size 4
embedding_layer = nn.Embedding(num_embeddings=10, embedding_dim=4)

# Example token indices in a batch (shape: batch_size=2, sequence_length=3)
token_ids = torch.tensor([
    [2, 5, 1],
    [0, 9, 3]
])

# Passing token IDs through the embedding layer
embeddings = embedding_layer(token_ids)

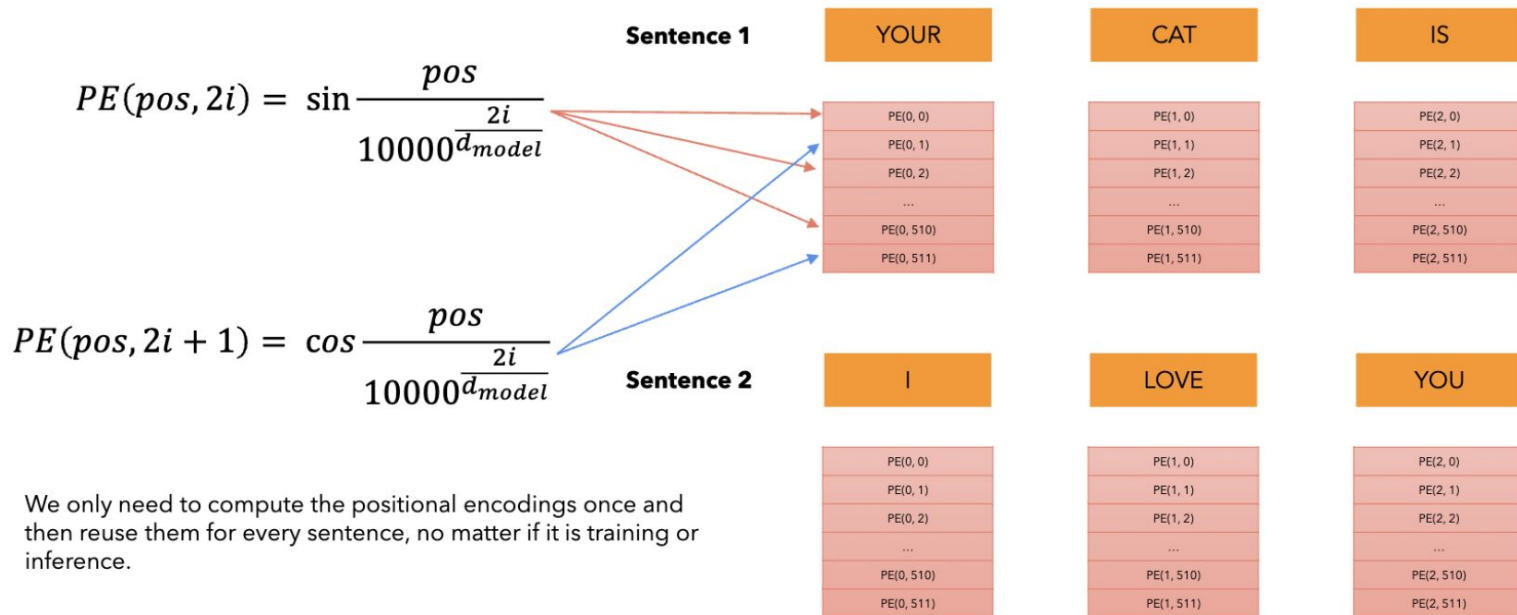
# embeddings now has shape: (2, 3, 4)
print("Embeddings shape:", embeddings.shape)
print("Embeddings:", embeddings)
```

The output vectors (embeddings) are learnable parameters—updated during training to capture semantic or contextual relationships.

What is positional encoding?

Original sentence	YOUR	CAT	IS	A	LOVELY	CAT
Embedding (vector of size 512)	952.207	171.411	621.659	776.562	6422.693	171.411
	5450.840	3276.350	1304.051	5567.288	6315.080	3276.350
	1853.448	9192.819	0.565	58.942	9358.778	9192.819
	...	...	...	...	...	...
	1.658	3633.421	7679.805	2716.194	2141.081	3633.421
	2671.529	8390.473	4506.025	5119.949	735.147	8390.473
Position Embedding (vector of size 512). Only computed once and reused for every sentence during training and inference.	+	+	+	+	+	+
	...	1664.068	...	...	...	1281.458
	...	8080.133	...	...	...	7902.890
	...	2620.399	...	...	...	912.970
	...	...	...	...	...	3821.102
	...	9386.405	...	...	...	1659.217
Encoder Input (vector of size 512)	=	=	=	=	=	=
	...	1835.479	...	...	...	1452.869
	...	11356.483	...	...	...	11179.24
	...	11813.218	...	...	...	10105.789
	...	...	...	...	...	...
	...	13019.826	...	...	...	5292.638
	...	11510.632	...	...	...	15409.093

What is positional encoding?



We only need to compute the positional encodings once and then reuse them for every sentence, no matter if it is training or inference.

PositionalEncoding

```
class PositionalEncoding(nn.Module):

    def __init__(self, d_model: int, seq_len: int, dropout: float) -> None:
        super().__init__()
        self.d_model = d_model
        self.seq_len = seq_len
        self.dropout = nn.Dropout(dropout)
        # Create a matrix of shape (seq_len, d_model)
        pe = torch.zeros(seq_len, d_model)
        # Create a vector of shape (seq_len)
        position = torch.arange(0, seq_len, dtype=torch.float).unsqueeze(1) # (seq_len, 1)
        # Create a vector of shape (d_model)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)) # (d_model / 2)
        # Apply sine to even indices
        pe[:, 0::2] = torch.sin(position * div_term) # sin(position * (10000 ** (2i / d_model))
        # Apply cosine to odd indices
        pe[:, 1::2] = torch.cos(position * div_term) # cos(position * (10000 ** (2i / d_model))
        # Add a batch dimension to the positional encoding
        pe = pe.unsqueeze(0) # (1, seq_len, d_model)
        # Register the positional encoding as a buffer
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + (self.pe[:, :x.shape[1], :]).requires_grad_(False) # (batch, seq_len, d_model)
        return self.dropout(x)
```

Explanations in Next slide

PositionalEncoding

python

 Copy  Edit

```
class PositionalEncoding(nn.Module):  
    def __init__(self, d_model: int, seq_len: int, dropout: float) -> None:  
        super().__init__()  
        self.d_model = d_model  
        self.seq_len = seq_len  
        self.dropout = nn.Dropout(dropout)
```

- `d_model` : The dimensionality of embeddings.
- `seq_len` : The maximum sequence length.
- `dropout` : A regularization technique to prevent overfitting.

Explanations in Next slide

PositionalEncoding

Step 1: Creating the Positional Encoding Matrix

python

Copy Edit

```
pe = torch.zeros(seq_len, d_model)
position = torch.arange(0, seq_len, dtype=torch.float).unsqueeze(1) # (seq_len, 1)
```

- `pe`: A zero tensor of shape `(seq_len, d_model)` to store positional encodings.
- `position`: A tensor with values `[0, 1, 2, ..., seq_len-1]`, representing token positions.

Explanations in Next slide

PositionalEncoding

Step 2: Computing Frequency Terms

python

Copy Edit

```
div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model
```

- The formula ensures different frequency scales for different dimensions.
- It computes the denominator $10000^{\frac{2i}{d_{\text{model}}}}$, which gradually decreases for higher `d_model` values.
- The values in `div_term` are used to create wave-like positional variations.

Explanations in Next slide

PositionalEncoding

Step 3: Applying Sine and Cosine Functions

python

Copy Edit

```
pe[:, 0::2] = torch.sin(position * div_term) # Sinusoidal function for even indices
pe[:, 1::2] = torch.cos(position * div_term) # Cosine function for odd indices
```

- **Even indices** of `d_model` use the sine function.
- **Odd indices** of `d_model` use the cosine function.

The **motivation** behind this sinusoidal approach:

- **Allows interpolation:** Works for longer sequences not seen during training.
- **Preserves uniqueness:** Each position has a unique encoding.
- **Encodes distances naturally:** Differences between encodings correspond to token distances.

Explanations in Next slide

PositionalEncoding

Step 4: Registering as a Buffer

python

Copy Edit

```
pe = pe.unsqueeze(0) # Shape becomes (1, seq_len, d_model)
self.register_buffer('pe', pe)
```

- `unsqueeze(0)`: Adds a batch dimension, making it `(1, seq_len, d_model)`.
- `register_buffer`: Stores `pe` inside the module but prevents it from being updated during training (it's a fixed constant).

Explanations in Next slide

PositionalEncoding

Forward Pass

python

Copy Edit

```
def forward(self, x):  
    x = x + (self.pe[:, :x.shape[1], :]).requires_grad_(False) # Add positional  
    return self.dropout(x)
```

- `self.pe[:, :x.shape[1], :]` : Selects the first `x.shape[1]` positions (in case of variable-length sequences).
- `x + self.pe[...]` : Adds positional encodings to the input embeddings.
- `requires_grad_(False)` : Prevents gradients from being computed for positional encodings.
- `self.dropout(x)` : Applies dropout for regularization.

Explanations in Next slide

PositionalEncoding

Example Usage

python

Copy Edit

```
import torch
from torch import nn
import math

# Define parameters
d_model = 16 # Embedding size
seq_len = 10 # Sequence length
dropout = 0.1 # Dropout probability

# Create the positional encoding layer
pos_encoding = PositionalEncoding(d_model, seq_len, dropout)

# Create a batch of input embeddings (batch_size=2, seq_len=10, d_model=16)
embeddings = torch.randn(2, seq_len, d_model)

# Apply positional encoding
encoded_embeddings = pos_encoding(embeddings)

print("Encoded Embeddings Shape:", encoded_embeddings.shape)
```

Explanations in Next slide

Encoder

Layer Normalization

Layer Normalization

What is layer normalization?

Batch of 3 items

ITEM 1

50.147
3314.825
...
...
8463.361
8.021

μ_1

σ_1^2

ITEM 2

1242.223
688.123
...
...
434.944
149.442

μ_2

σ_2^2

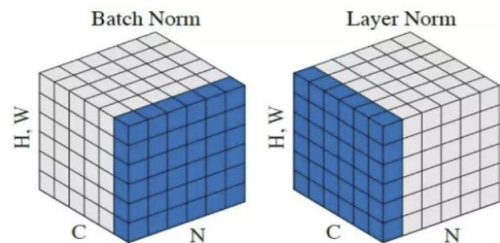
ITEM 3

9.370
4606.674
...
...
944.705
21189.444

μ_3

σ_3^2

$$\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$



We also introduce two parameters, usually called **gamma** (multiplicative) and **beta** (additive) that introduce some fluctuations in the data, because maybe having all values between 0 and 1 may be too restrictive for the network. The network will learn to tune these two parameters to introduce fluctuations when necessary.

Layer Normalization

You, 41 minutes ago | 1 author (You)

```
class LayerNormalization(nn.Module):

    def __init__(self, features: int, eps:float=10**-6) -> None:
        super().__init__()
        self.eps = eps
        self.alpha = nn.Parameter(torch.ones(features)) # alpha is a learnable parameter
        self.bias = nn.Parameter(torch.zeros(features)) # bias is a learnable parameter

    def forward(self, x):
        # x: (batch, seq_len, hidden_size)
        # Keep the dimension for broadcasting
        mean = x.mean(dim = -1, keepdim = True) # (batch, seq_len, 1)
        # Keep the dimension for broadcasting
        std = x.std(dim = -1, keepdim = True) # (batch, seq_len, 1)
        # eps is to prevent dividing by zero or when std is very small
        return self.alpha * (x - mean) / (std + self.eps) + self.bias
```

Explanations in Next slide

Layer Normalization

`__init__` Method

python

 Copy  Edit

```
def __init__(self, features: int, eps: float = 1e-6) -> None:
    super().__init__()
    self.eps = eps
    self.alpha = nn.Parameter(torch.ones(features)) # Learnable scaling factor
    self.bias = nn.Parameter(torch.zeros(features)) # Learnable shift factor
```

- `features` : The number of hidden dimensions (e.g., `d_model` in Transformers).
- `eps` : A small constant (`1e-6`) to **prevent division by zero** when computing standard deviation.
- `alpha` : **Learnable** scale factor (like gamma in BatchNorm).
- `bias` : **Learnable** shift factor (like beta in BatchNorm).

Explanations in Next slide

Layer Normalization

forward Method

python

Copy Edit

```
def forward(self, x):  
    mean = x.mean(dim=-1, keepdim=True) # Mean across the last dimension (hidden size)  
    std = x.std(dim=-1, keepdim=True) # Standard deviation across the last dimension  
    return self.alpha * (x - mean) / (std + self.eps) + self.bias
```

📌 Steps:

1. Compute the **mean** across the last dimension (`hidden_size`).
2. Compute the **standard deviation** across the last dimension.
3. Normalize:

$$\text{normalized_x} = \frac{x - \text{mean}}{\text{std} + \epsilon}$$

4. Scale and shift using **learnable** parameters `alpha` (scale) and `bias` (shift).

Explanations in Next slide

Layer Normalization

Key Differences: LayerNorm vs. BatchNorm

Feature	LayerNorm	BatchNorm
Normalizes across	Feature dimension (<code>hidden_size</code>)	Batch dimension
Dependency on batch size?	 No	 Yes
Used in	Transformers (NLP)	CNNs (Computer Vision)
Computation	Per token	Across batch

 **LayerNorm is better for Transformers** since it ensures that each token's representation is independently normalized, making it **batch-independent**.

Explanations in Next slide

Encoder

Feed Forward Block

Feed Forward Block

Understanding the FeedForward Block in Transformers

The **FeedForward Block (FFN)** is a crucial component of Transformer models, providing non-linearity and allowing the model to **learn richer representations**. It operates **independently** on each token in the sequence.

Why is the FeedForward Block Needed?

- 1 Introduces non-linearity using ReLU activation.
- 2 Projects features into a higher-dimensional space (`d_ff`) and back to `d_model` .
- 3 Enhances expressivity by allowing richer transformations on token embeddings.
- 4 Dropout helps prevent overfitting.

Explanations in Next slide

Feed Forward Block

You, 46 minutes ago | 1 author (You)

```
class FeedForwardBlock(nn.Module):
```

```
    def __init__(self, d_model: int, d_ff: int, dropout: float) -> None:
```

```
        super().__init__()
```

```
        self.linear_1 = nn.Linear(d_model, d_ff) # w1 and b1
```

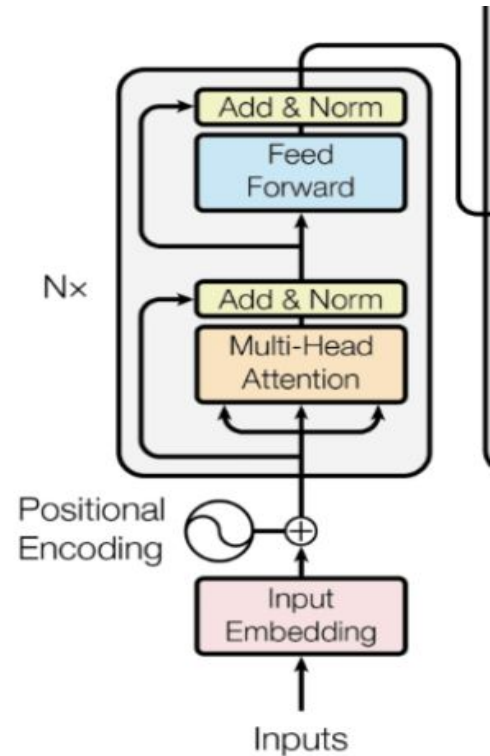
```
        self.dropout = nn.Dropout(dropout)
```

```
        self.linear_2 = nn.Linear(d_ff, d_model) # w2 and b2
```

```
    def forward(self, x):
```

```
        # (batch, seq_len, d_model) --> (batch, seq_len, d_ff) --> (batch, seq_len, d_model)
```

```
        return self.linear_2(self.dropout(torch.relu(self.linear_1(x))))
```



Explanations in Next slide

Feed Forward Block

Breaking Down the Code

`__init__` Method

python

 Copy  Edit

```
def __init__(self, d_model: int, d_ff: int, dropout: float) -> None:
    super().__init__()
    self.linear_1 = nn.Linear(d_model, d_ff) # w1 and b1
    self.dropout = nn.Dropout(dropout)
    self.linear_2 = nn.Linear(d_ff, d_model) # w2 and b2
```

- `d_model`: The embedding size (same as the input dimension).
- `d_ff`: The **intermediate (expanded) size** (typically `d_ff = 4 * d_model`).
- `dropout`: Regularization to prevent overfitting.
- **Two Linear Layers:**
 - `linear_1`: Expands dimension (`d_model → d_ff`).
 - `linear_2`: Projects back (`d_ff → d_model`).

Explanations in Next slide

Feed Forward Block

forward Method

python

Copy Edit

```
def forward(self, x):  
    # (batch, seq_len, d_model) --> (batch, seq_len, d_ff) --> (batch, seq_len, d_model)  
    return self.linear_2(self.dropout(torch.relu(self.linear_1(x))))
```

🔴 Step-by-step explanation:

1. First linear transformation:

$$x' = W_1x + b_1$$

Expands the feature dimension:

`(batch, seq_len, d_model) → (batch, seq_len, d_ff)`

2. Apply ReLU activation:

$$x' = \text{ReLU}(x')$$

Adds non-linearity.

3. Apply Dropout:

$$x' = \text{Dropout}(x')$$

Helps with regularization.

4. Second linear transformation:

$$x'' = W_2x' + b_2$$

Projects back to the original feature dimension:

`(batch, seq_len, d_ff) → (batch, seq_len, d_model)`

Explanations in Next slide

Feed Forward Block

Example Usage

python

 Copy  Edit

```
import torch

ffn = FeedForwardBlock(d_model=512, d_ff=2048, dropout=0.1)

# Example input (batch_size=2, seq_len=5, hidden_size=512)
x = torch.rand(2, 5, 512)

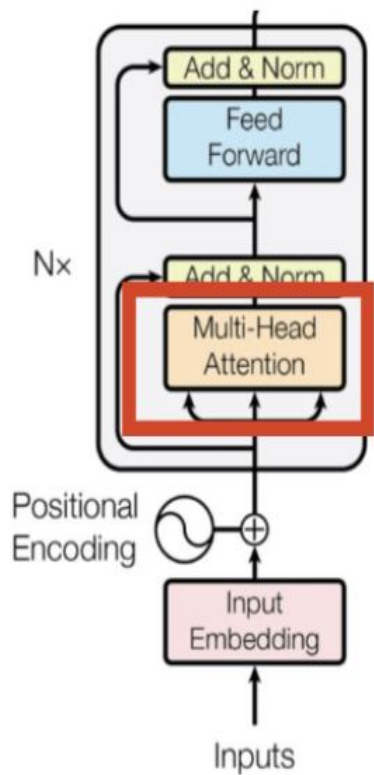
output = ffn(x)
print(output.shape) # Output: torch.Size([2, 5, 512])
```

◆ Shape remains the same ((batch, seq_len, d_model)) – the transformation happens internally.

Intuition Behind `d_ff`

- `d_ff` is typically 4x larger than `d_model` ($d_{ff} = 4 * d_{model}$).
- The expansion allows richer transformations before projecting back.
- Similar to the hidden layers in deep MLPs.

Explanations in Next slide



Encoder

Multi-Head Attention

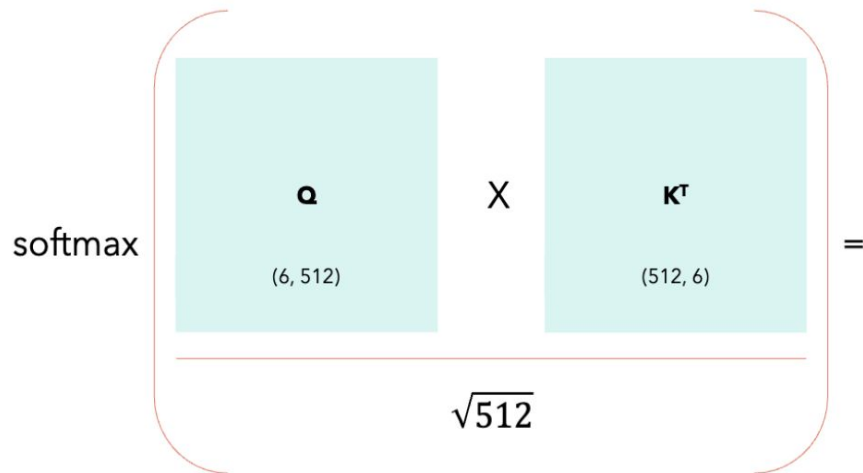
What is Self-Attention?

Self-Attention allows the model to relate words to each other.

In this simple case we consider the sequence length **seq** = 6 and **d_{model}** = **d_k** = 512.

The matrices **Q**, **K** and **V** are just the input sentence.

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

* all values are random.

* for simplicity I considered only one head, which makes $d_{\text{model}} = d_k$.

(6, 6)

How to compute Self-Attention?

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

(6, 6)

X

V

(6, 512)

=

Attention

(6, 512)

Each row in this matrix captures not only the meaning (given by the embedding) or the position in the sentence (represented by the positional encodings) but also each word's interaction with other words.

Self-Attention in detail

- Self-Attention is permutation invariant.
- Self-Attention requires no parameters. Up to now the interaction between words has been driven by their embedding and the positional encodings. This will change later.
- We expect values along the diagonal to be the highest.
- If we don't want some positions to interact, we can always set their values to $-\infty$ before applying the *softmax* in this matrix and the model will not learn those interactions. We will use this in the decoder.

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

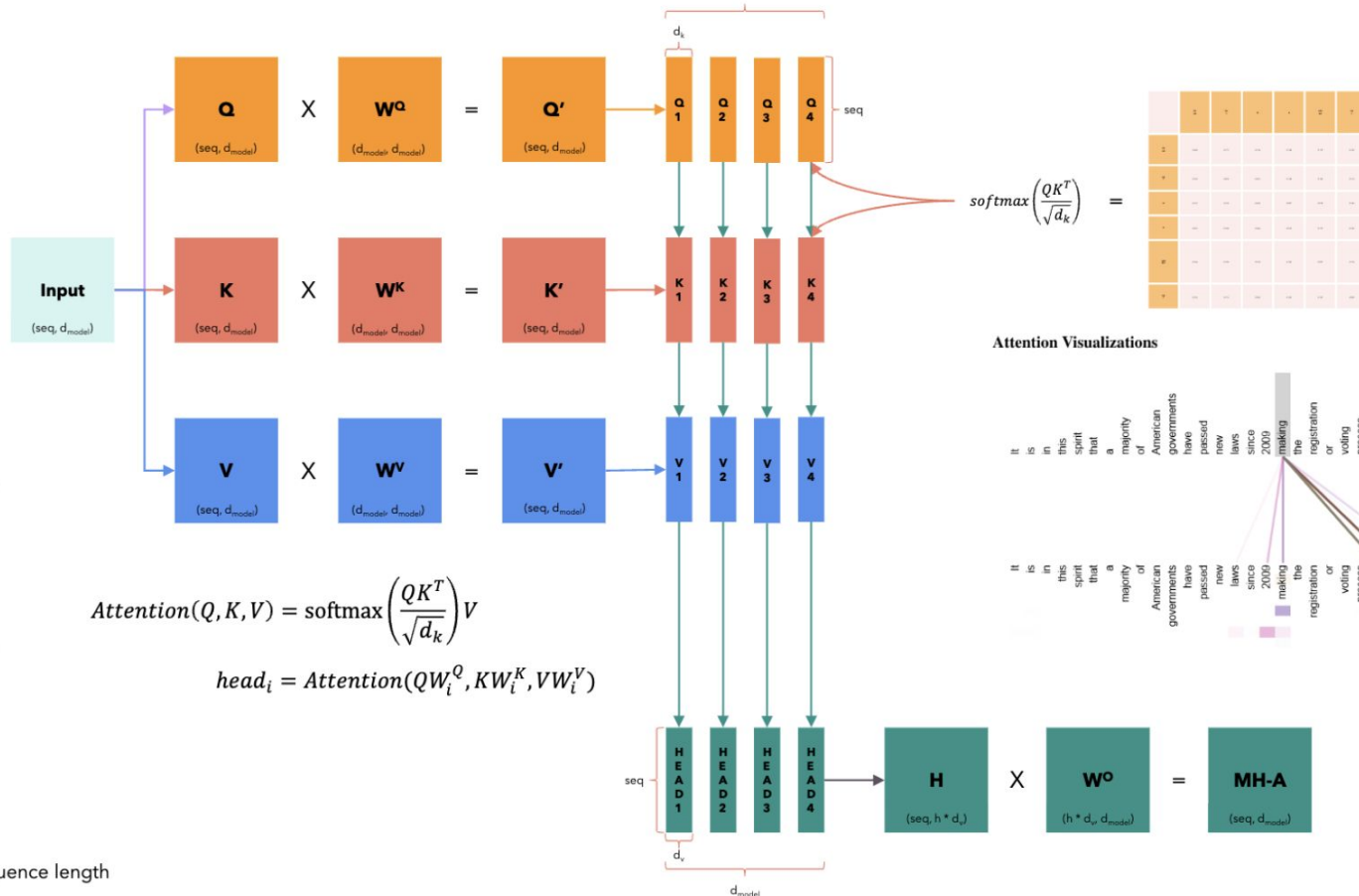
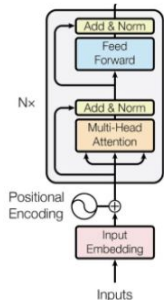
Umar Jamil - <https://github.com/hkproj/transformer-from-scratch-notes>

Explanations in Next slide

Multi-head Attention

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$MultiHead(Q, K, V) = \text{Concat}(head_1 \dots head_h)W^O$$
$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

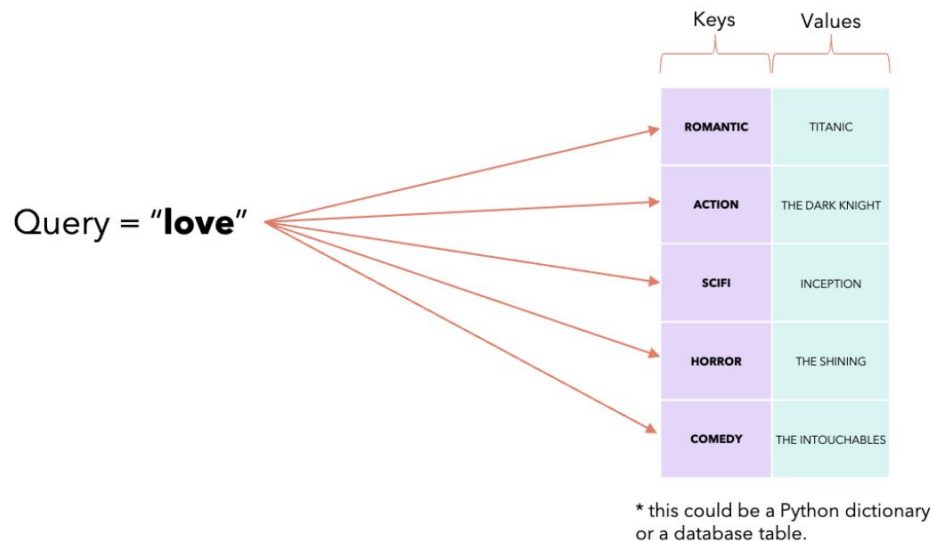


seq = sequence length
 d_{model} = size of the embedding vector
 h = number of heads
 $d_k = d_v$ = d_{model} / h

$$MultiHead(Q, K, V) = \text{Concat}(\text{head}_1 \dots \text{head}_h)W^O$$

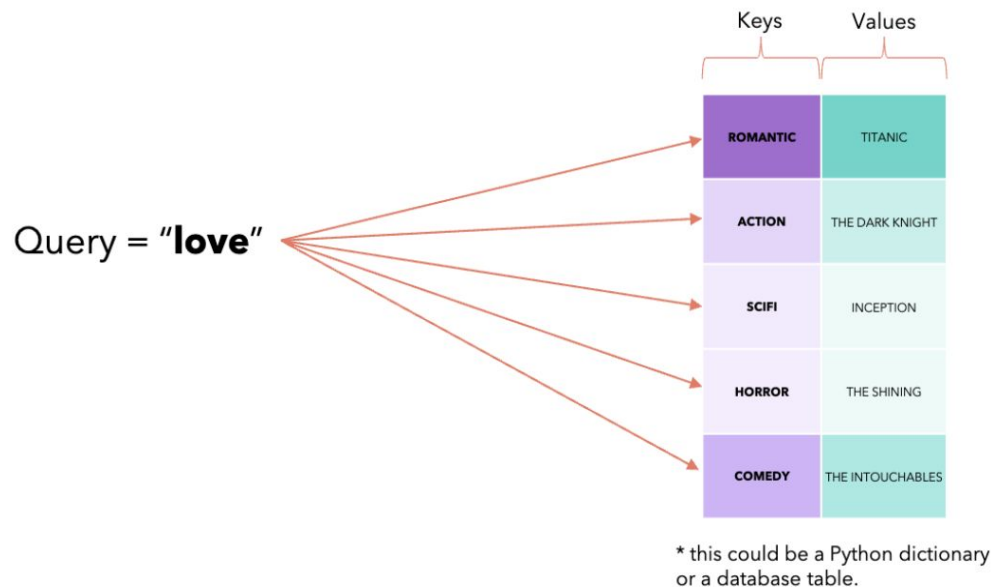
Why query, keys and values?

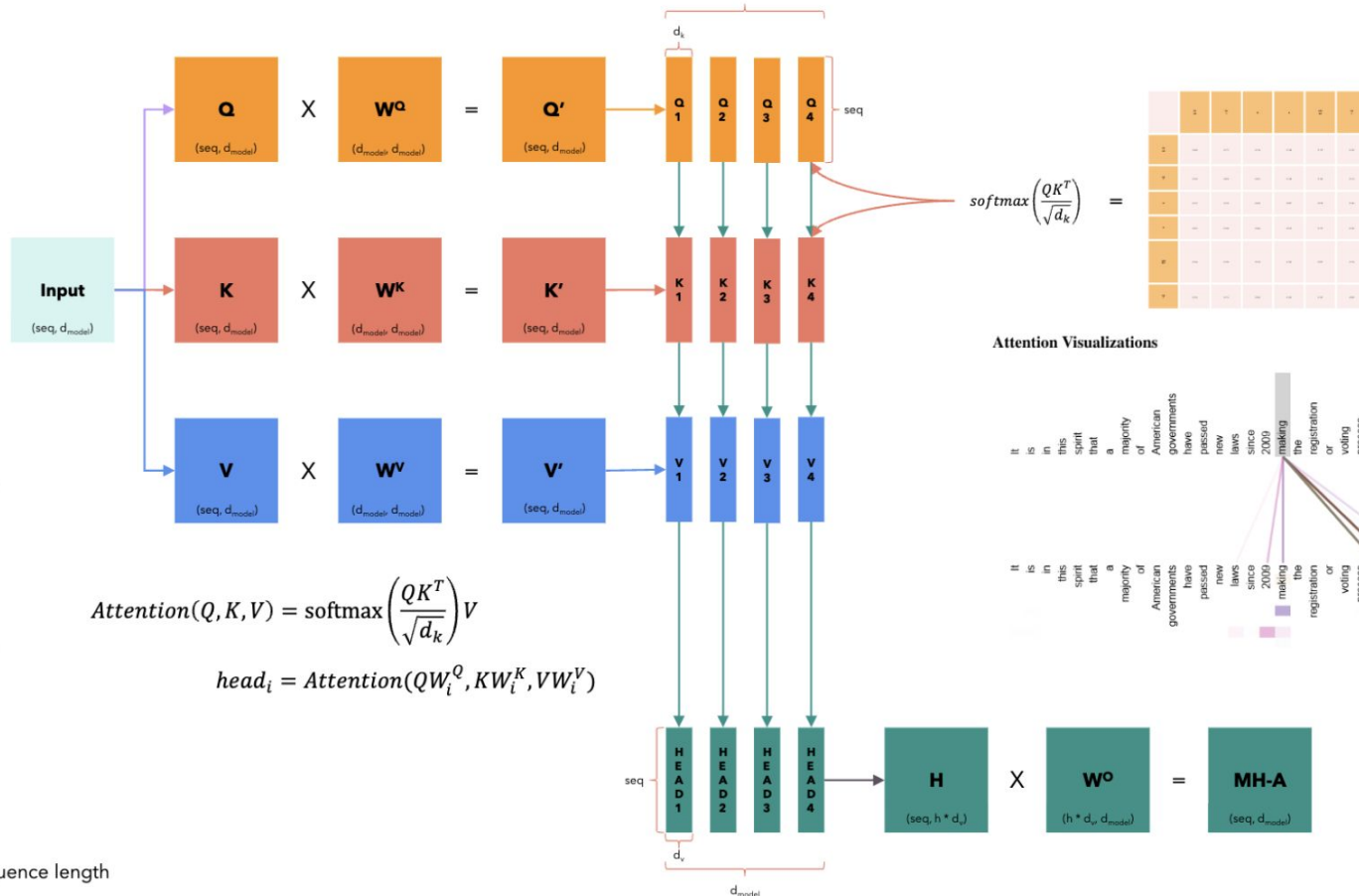
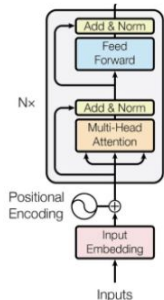
The Internet says that these terms come from the database terminology or the Python-like dictionaries.



Why query, keys and values?

The Internet says that these terms come from the database terminology or the Python-like dictionaries.





seq = sequence length
 d_{model} = size of the embedding vector
 h = number of heads
 $d_k = d_v$ = d_{model} / h

$$MultiHead(Q, K, V) = \text{Concat}(\text{head}_1 \dots \text{head}_h)W^O$$

Multi-Head Attention

```
class MultiHeadAttentionBlock(nn.Module):
```

```
    def __init__(self, d_model: int, h: int, dropout: float) -> None:
```

```
        super().__init__()
```

```
        self.d_model = d_model # Embedding vector size
```

```
        self.h = h # Number of heads
```

```
        # Make sure d_model is divisible by h
```

```
        assert d_model % h == 0, "d_model is not divisible by h"
```

```
        self.d_k = d_model // h # Dimension of vector seen by each head
```

```
        self.w_q = nn.Linear(d_model, d_model, bias=False) # Wq
```

```
        self.w_k = nn.Linear(d_model, d_model, bias=False) # Wk
```

```
        self.w_v = nn.Linear(d_model, d_model, bias=False) # Wv
```

```
        self.w_o = nn.Linear(d_model, d_model, bias=False) # Wo
```

```
        self.dropout = nn.Dropout(dropout)
```

```
    @staticmethod
```

```
    def attention(query, key, value, mask, dropout: nn.Dropout):
```

```
        d_k = query.shape[-1]
```

```
        # Just apply the formula from the paper
```

```
        # (batch, h, seq_len, d_k) --> (batch, h, seq_len, seq_len)
```

```
        attention_scores = (query @ key.transpose(-2, -1)) / math.sqrt(d_k)
```

```
        if mask is not None:
```

```
            # Write a very low value (indicating -inf) to the positions where mask == 0
```

```
            attention_scores.masked_fill_(mask == 0, -1e9)
```

```
        attention_scores = attention_scores.softmax(dim=-1) # (batch, h, seq_len, seq_len) # Apply softmax
```

```
        if dropout is not None:
```

```
            attention_scores = dropout(attention_scores)
```

```
        # (batch, h, seq_len, seq_len) --> (batch, h, seq_len, d_k)
```

```
        # return attention scores which can be used for visualization
```

```
        return (attention_scores @ value), attention_scores
```

```
    def forward(self, q, k, v, mask):
```

```
        query = self.w_q(q) # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
```

```
        key = self.w_k(k) # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
```

```
        value = self.w_v(v) # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
```

```
        # (batch, seq_len, d_model) --> (batch, seq_len, h, d_k) --> (batch, h, seq_len, d_k)
```

```
        query = query.view(query.shape[0], query.shape[1], self.h, self.d_k).transpose(1, 2)
```

```
        key = key.view(key.shape[0], key.shape[1], self.h, self.d_k).transpose(1, 2)
```

```
        value = value.view(value.shape[0], value.shape[1], self.h, self.d_k).transpose(1, 2)
```

```
        # Calculate attention
```

```
        x, self.attention_scores = MultiHeadAttentionBlock.attention(query, key, value, mask, self.dropout)
```

```
        # Combine all the heads together
```

```
        # (batch, h, seq_len, d_k) --> (batch, seq_len, h, d_k) --> (batch, seq_len, d_model)
```

```
        x = x.transpose(1, 2).contiguous().view(x.shape[0], -1, self.h * self.d_k)
```

```
        # Multiply by Wo
```

```
        # (batch, seq_len, d_model) --> (batch, seq_len, d_model)
```

```
        return self.w_o(x)
```

You, 60 minutes ago • Transformer Implementation from Umar Jamil

Multi-Head Attention

`__init__` Method

python

Copy Edit

```
def __init__(self, d_model: int, h: int, dropout: float) -> None:
    super().__init__()
    self.d_model = d_model # Embedding vector size
    self.h = h # Number of heads
    assert d_model % h == 0, "d_model is not divisible by h"

    self.d_k = d_model // h # Dimension of vector seen by each head
    self.w_q = nn.Linear(d_model, d_model, bias=False) # Wq
    self.w_k = nn.Linear(d_model, d_model, bias=False) # Wk
    self.w_v = nn.Linear(d_model, d_model, bias=False) # Wv
    self.w_o = nn.Linear(d_model, d_model, bias=False) # Wo
    self.dropout = nn.Dropout(dropout)
```

🔴 Step-by-step explanation:

- `d_model`: The embedding size.
- `h`: Number of attention heads.
- `d_k`: Each head gets a `d_k = d_model / h` size vector.
- **Weight matrices** (`Wq`, `Wk`, `Wv`, `Wo`):
 - Each head gets a **separate transformation** for **queries (Q)**, **keys (K)**, and **values (V)**.
 - `Wo` is used to combine outputs from different heads.
- **Dropout** is applied for regularization.

Explanations in Next slide

Multi-Head Attention

Attention Calculation

python

Copy Edit

```
@staticmethod
def attention(query, key, value, mask, dropout: nn.Dropout):
    d_k = query.shape[-1]
    attention_scores = (query @ key.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        attention_scores.masked_fill_(mask == 0, -1e9)
    attention_scores = attention_scores.softmax(dim=-1)
    if dropout is not None:
        attention_scores = dropout(attention_scores)
    return (attention_scores @ value), attention_scores
```

🔗 Step-by-step explanation:

1. Compute Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- The division by `sqrt(d_k)` prevents extremely large values, improving stability.

2. Apply masking (optional):

- Used in **decoder layers** to prevent the model from attending to future tokens.

3. Apply softmax:

- Converts scores into a **probability distribution**.

4. Dropout for regularization.

Explanations in next slide

Multi-Head Attention

Forward Pass

```
python Copy Edit

def forward(self, q, k, v, mask):
    query = self.w_q(q)
    key = self.w_k(k)
    value = self.w_v(v)

    query = query.view(query.shape[0], query.shape[1], self.h, self.d_k).transpose(1, 2)
    key = key.view(key.shape[0], key.shape[1], self.h, self.d_k).transpose(1, 2)
    value = value.view(value.shape[0], value.shape[1], self.h, self.d_k).transpose(1, 2)

    x, self.attention_scores = MultiHeadAttentionBlock.attention(query, key, value, mask)

    x = x.transpose(1, 2).contiguous().view(x.shape[0], -1, self.h * self.d_k)

    return self.w_o(x)
```

🔗 Step-by-step explanation:

1. Project inputs using `Wq`, `Wk`, `Wv`:
 - Transform `query`, `key`, and `value` using `nn.Linear` layers.
2. Reshape into multiple heads:
 - `dmodel` → `(h, dk)`, then rearrange dimensions.
3. Compute attention scores using the `attention` method.
4. Concatenate all heads:
 - Reshape back from multiple heads to a single vector.
5. Apply the final projection `Wo`.

Encoder

Residual Connection

Residual Connection

```
class ResidualConnection(nn.Module):  
  
    def __init__(self, features: int, dropout: float) -> None:  
        super().__init__()  
        self.dropout = nn.Dropout(dropout)  
        self.norm = LayerNormalization(features)  
  
    def forward(self, x, sublayer):  
        return x + self.dropout(sublayer(self.norm(x)))
```

📌 Step-by-step explanation:

- `self.norm = LayerNormalization(features):`
 - Applies **layer normalization** before the sublayer.
- `self.dropout = nn.Dropout(dropout):`
 - Regularizes by randomly setting activations to 0 during training.
- `forward(self, x, sublayer):`
 - `self.norm(x)` → Normalizes input.
 - Passes **normalized input** to `sublayer(x)`.
 - Applies **dropout** after the sublayer operation.
 - Adds the **residual connection** (`x + ...`).

Explanations in Next slide

Encoder

Encoder and EncoderBlock

Encoder

```
class EncoderBlock(nn.Module):

    def __init__(self, features: int, self_attention_block: MultiHeadAttentionBlock, feed_forward_block: FeedForwardBlock, dropout: float) -> None:
        super().__init__()
        self.self_attention_block = self_attention_block
        self.feed_forward_block = feed_forward_block
        self.residual_connections = nn.ModuleList([ResidualConnection(features, dropout) for _ in range(2)])

    def forward(self, x, src_mask):
        x = self.residual_connections[0](x, lambda x: self.self_attention_block(x, x, x, src_mask))
        x = self.residual_connections[1](x, self.feed_forward_block)
        return x
```

You, 1 hour ago | 1 author (You)

```
class Encoder(nn.Module):

    def __init__(self, features: int, layers: nn.ModuleList) -> None:
        super().__init__()
        self.layers = layers
        self.norm = LayerNormalization(features)

    def forward(self, x, mask):
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)
```

You, 1 hour ago • Transformer Implementation from Umar Jamil

Explanations in Next slide

Encoder

1 Multi-Head Self-Attention Block

- Applies `MultiHeadAttentionBlock` to `x` using a **residual connection**.
- Uses a lambda function: `lambda x: self.self_attention_block(x, x, x, src_mask)`, which passes `x` as query, key, and value.
- **Mask** (`src_mask`) is used to prevent attending to padding tokens.

2 Feed-Forward Network

- Passes the output through a **Feed-Forward Layer** (`FeedForwardBlock`).
- Another **Residual Connection** ensures stable learning.

Explanations in Next slide

Encoder

The **Encoder** is a stack of `EncoderBlock` layers.

Code Breakdown

python

 Copy  Edit

```
class Encoder(nn.Module):

    def __init__(self, features: int, layers: nn.ModuleList) -> None:
        super().__init__()
        self.layers = layers
        self.norm = LayerNormalization(features)

    def forward(self, x, mask):
        for layer in self.layers:
            x = layer(x, mask) # Pass input through each encoder block
        return self.norm(x) # Apply final Layer Normalization
```

Step-by-Step Execution

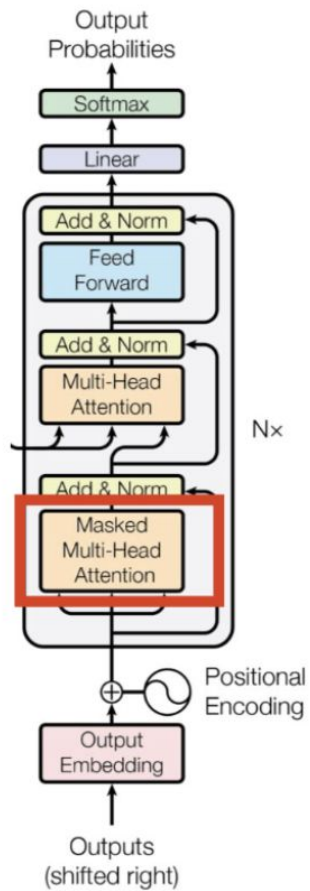
Iterates over `layers` :

- The input `x` is passed through each `EncoderBlock` in the stack.

Final Layer Normalization:

- Ensures consistent activations and stability.
- Prevents vanishing/exploding gradients.

Explanations in Next slide



Decoder

Decoder

What is Masked Multi-Head Attention?

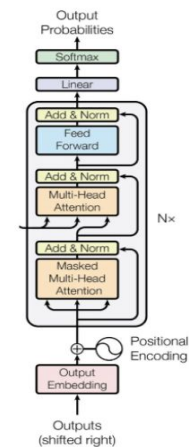
Our goal is to make the model causal: it means the output at a certain position can only depend on the words on the previous positions. The model **must not** be able to see future words.

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

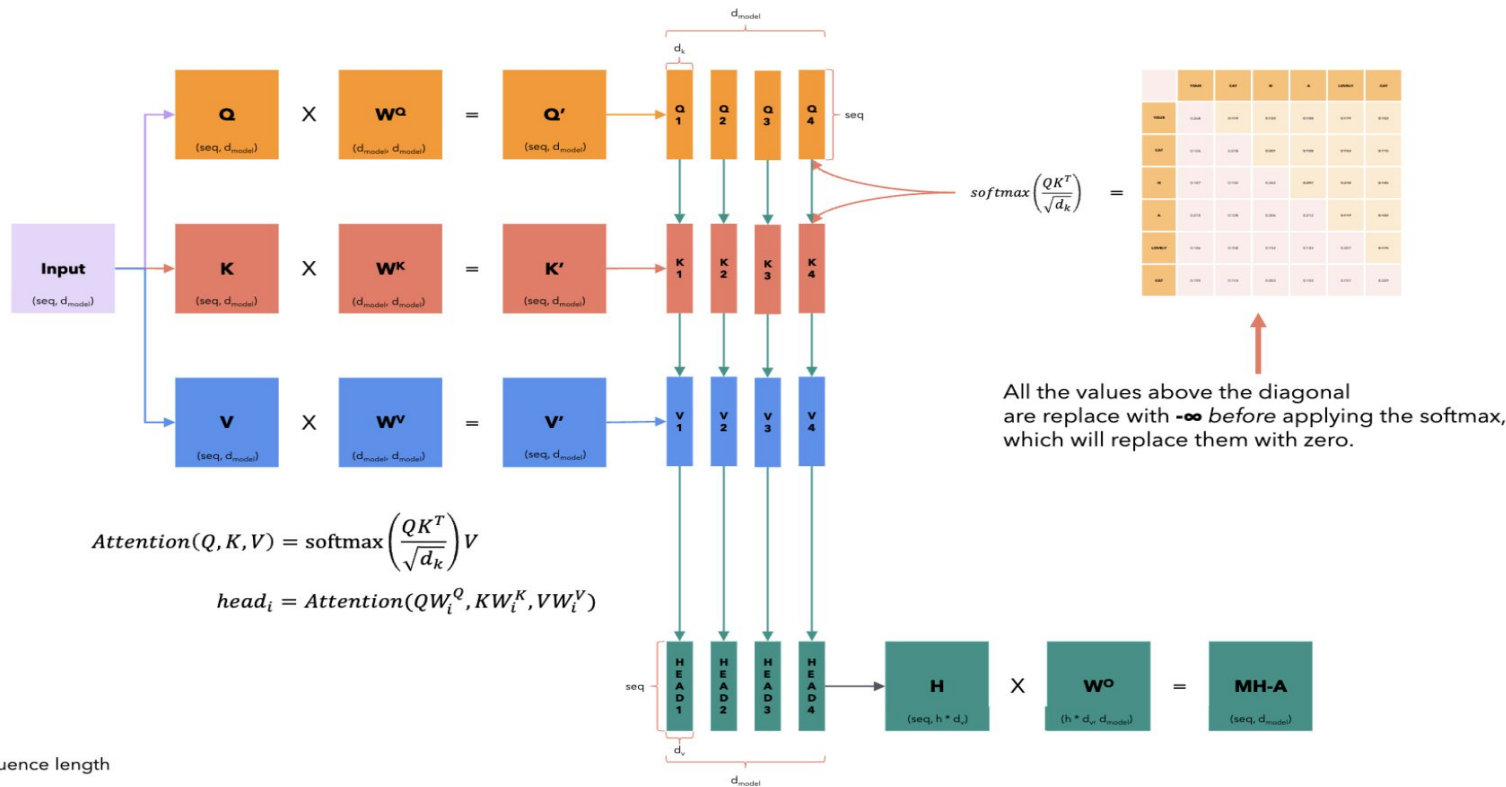
Umar Jamil - <https://github.com/hkproj/transformer-from-scratch-notes>

Explanations in Next slide

Decoder



seq = sequence length
 d_{model} = size of the embedding vector
 h = number of heads
 $d_k = d_v = d_{model} / h$



Decoder

```
class DecoderBlock(nn.Module):

    def __init__(self, features: int,
                 self_attention_block: MultiHeadAttentionBlock,
                 cross_attention_block: MultiHeadAttentionBlock,
                 feed_forward_block: FeedForwardBlock,
                 dropout: float) -> None:
        super().__init__()
        self.self_attention_block = self_attention_block
        self.cross_attention_block = cross_attention_block
        self.feed_forward_block = feed_forward_block
        self.residual_connections = nn.ModuleList([ResidualConnection(features, dropout) for _ in range(3)])

    def forward(self, x, encoder_output, src_mask, tgt_mask):
        x = self.residual_connections[0](x, lambda x: self.self_attention_block(x, x, x, tgt_mask))
        x = self.residual_connections[1](x, lambda x: self.cross_attention_block(x, encoder_output, encoder_output, src_mask))
        x = self.residual_connections[2](x, self.feed_forward_block)
        return x
```

You, 1 hour ago | 1 author (You)

```
class Decoder(nn.Module):

    def __init__(self, features: int, layers: nn.ModuleList) -> None:
        super().__init__()
        self.layers = layers
        self.norm = LayerNormalization(features)

    def forward(self, x, encoder_output, src_mask, tgt_mask):
        for layer in self.layers:
            x = layer(x, encoder_output, src_mask, tgt_mask)
        return self.norm(x)
```

Explanations in Next slide

Decoder

Step-by-Step Execution

1 Masked Multi-Head Self-Attention

- Similar to the encoder's self-attention but **uses a causal mask** (`tgt_mask`) to prevent attending to future tokens.
- This ensures **autoregressive decoding**.

2 Cross-Attention with Encoder Output

- Each token attends to **encoder outputs** using `MultiHeadAttentionBlock(x, encoder_output, encoder_output, src_mask)`.
- Allows the decoder to focus on **relevant input information**.

3 Feed-Forward Network

- Applies a simple **MLP transformation** to enhance representations.

4 Residual Connections

- Each sublayer has a **Residual Connection + Layer Normalization**.

Explanations in Next slide

Decoder

Projection Layer

Decoder

You, 1 hour ago | 1 author (You)

```
class ProjectionLayer(nn.Module):  
  
    def __init__(self, d_model, vocab_size) -> None:  
        super().__init__()  
        self.proj = nn.Linear(d_model, vocab_size)  
  
    def forward(self, x) -> None:  
        # (batch, seq_len, d_model) --> (batch, seq_len, vocab_size)  
        return self.proj(x)
```

Explanations in Next slide

Transformer

Transformer

```
class Transformer(nn.Module):

    def __init__(self, encoder: Encoder,
                 decoder: Decoder,
                 src_embed: InputEmbeddings,
                 tgt_embed: InputEmbeddings,
                 src_pos: PositionalEncoding,
                 tgt_pos: PositionalEncoding,
                 projection_layer: ProjectionLayer) -> None:
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.src_embed = src_embed
        self.tgt_embed = tgt_embed
        self.src_pos = src_pos
        self.tgt_pos = tgt_pos
        self.projection_layer = projection_layer

    def encode(self, src, src_mask):
        # (batch, seq_len, d_model)
        src = self.src_embed(src)
        src = self.src_pos(src)
        return self.encoder(src, src_mask)

    def decode(self, encoder_output: torch.Tensor, src_mask: torch.Tensor, tgt: torch.Tensor, tgt_mask: torch.Tensor):
        # (batch, seq_len, d_model)
        tgt = self.tgt_embed(tgt)
        tgt = self.tgt_pos(tgt)
        return self.decoder(tgt, encoder_output, src_mask, tgt_mask)

    def project(self, x):
        # (batch, seq_len, vocab_size)
        return self.projection_layer(x)
```

Explanations in Next slide

Transformer

Key Takeaways

✓ Transformer Components Work Together:

- **Encoder:** Processes the source sentence.
- **Decoder:** Generates the output step-by-step.
- **Embedding + Positional Encoding:** Provides rich token representations.
- **Projection Layer:** Maps decoder output to the vocabulary.

✓ Masking is Critical:

- **Padding Mask** (`src_mask`) prevents attention to padding tokens.
- **Causal Mask** (`tgt_mask`) ensures autoregressive decoding.

✓ Final Output:

- A **logit tensor** of shape (`batch`, `seq_len`, `vocab_size`), used for **next-token prediction**.

Explanations in Next slide

Transformer

Encoder Function

python

 Copy  Edit

```
def encode(self, src, src_mask):  
    # (batch, seq_len, d_model)  
    src = self.src_embed(src) # Convert tokens to embeddings  
    src = self.src_pos(src) # Add positional encoding  
    return self.encoder(src, src_mask) # Process with encoder
```

Step-by-Step

- 1 **Token Embeddings:** `src_embed(src)` ➡ Converts tokens into dense vectors.
- 2 **Positional Encoding:** `src_pos(src)` ➡ Adds positional information.
- 3 **Pass Through Encoder:** `self.encoder(src, src_mask)` ➡ Outputs contextualized representations.

Explanations in Next slide

Transformer

Decoder Function

python

 Copy  Edit

```
def decode(self, encoder_output: torch.Tensor, src_mask: torch.Tensor, tgt: torch.Tensor,
            # (batch, seq_len, d_model)
            tgt_mask: torch.Tensor) # Convert target tokens to embeddings
    tgt = self.tgt_embed(tgt)
    tgt = self.tgt_pos(tgt) # Add positional encoding
    return self.decoder(tgt, encoder_output, src_mask, tgt_mask) # Process with decoder
```

Step-by-Step

- 1 Convert target tokens to embeddings 
- 2 Add positional encoding 
- 3 Pass through decoder along with encoder output 

Explanations in Next slide

Transformer

Projection Layer

python

 Copy  Edit

```
def project(self, x):  
    # (batch, seq_len, vocab_size)  
    return self.projection_layer(x)
```

What It Does?

- The **decoder outputs a sequence of vectors** of shape `(batch, seq_len, d_model)`.
- The `projection_layer` **maps these vectors to vocabulary logits** of shape `(batch, seq_len, vocab_size)`.
- These logits are later passed to **softmax** for probability distribution over words.

Explanations in Next slide